Challenge 1 – AI Security

Title: Analysis and Security Hardening of an AI-Assisted Access Control API
Team: Team03
Verification Date: October 25, 2026
Scope: Local Python laboratory only

1. Introduction

The laboratory simulates a Flask API that receives an image, calls a facial recognition model and an object detection model, and then decides whether physical access should be granted. The provided facial recognition model always returns AUTORIZED with a confidence score of 0.98. The detector returns, among other results, KNIFE with a confidence score of 0.88, making it possible to test a security decision without using real-world data or systems.

The analysis follows the PROVE → DEFEND → RETEST cycle. The results in this document are limited to executions actually performed within the project directory.

2. Scenario and Security Objective

The primary asset is the access decision. Secondary assets include the confidentiality of the models and their structure, the integrity of the logging directory, service availability, and the confidentiality of diagnostic information.

A secure decision must deny access when an image is invalid, a model fails, an output is ambiguous, or a dangerous object is detected.

3. Architecture and Attack Surface

```
[local client / curl / test_client]
|
| multipart image (untrusted)
[Flask POST /api/access] -- T1 upload/HTTP boundary
|
decode + preprocess
|
[face model + object model]
|
fail-closed decision
|
[minimal JSON] + [logs/secure_uploads]
```

Untrusted inputs include the HTTP method, request size, filename, image bytes, and model outputs. The original code, models with known fingerprints, and decision policy are trusted assets.

4. Model Preparation and Compatibility

The dependencies available in .venv were verified: Python 3.14.7, NumPy 2.5.4, Flask 3.1.3, and OpenCV 5.0.0.93.

Both pickle files are 55 bytes and reference __main__.MockFaceRecognitionModel or __main__.MockObjectDetectionModel.

A direct import of lab_access_api produced the error:

AttributeError: module '__main__' has no attribute 'MockFaceRecognitionModel'

Running the file as a script is the execution mode expected by the model generator. The test runner therefore loads the baseline under __main__ to reproduce the laboratory environment without modifying the models.

The secured version retains the local classes, verifies their SHA-256 hashes, and then uses a RestrictedModelUnpickler limited to the two known classes.

5. Red Team Analysis of the Original Code

The main weaknesses are:

1. preprocess_image calls cv2.imdecode without validating the buffer or the result, then returns a raw image without resizing, normalization, or a batch dimension.
2. The route calls file.save() and then file.read(). The stream is already at its end, so OpenCV receives empty bytes.
3. The policy only checks for the weapon label, whereas the model returns knife. The threshold of 0.15 is not appropriate for a physical access control barrier.
4. The filename supplied by the client is concatenated with ./logs/.
5. pickle.load() is unrestricted.
6. The response exposes confidence scores, detected objects, the model path, IP address, version, and log path.
7. There is no request size limit, and app.run(..., debug=True) enables debugging when the application starts.
8. Exceptions are not converted into controlled errors, and the decision is not explicitly fail-closed.

The vulnerability matrix and their impacts are summarized in the table above and in the JSON evidence preserved under evidence/.

6. Controlled Red Team Tests

The latest evidence is stored in evidence/baseline_controlled_run_final.json. Earlier raw HTTP evidence is preserved in evidence/first_test.txt, evidence/first_error.txt, and evidence/second_error.txt.

Test A — POST without an image

- Baseline result: HTTP 400.
- Response: {"error":"No image provided"}.

Test B — Synthetic 300 × 300 PNG

- Baseline result: Generic HTTP 500 when using the test client.
- Earlier HTTP evidence shows the OpenCV error caused by an empty buffer.

Test C — Text file

- Baseline result: The same OpenCV HTTP 500 error, instead of a controlled rejection.

Preprocessing test

- A direct call using a valid PNG produced an image with shape (300,300,3) and type uint8.
- The model rejected this input shape.

Isolated policy test

- The expected tensor was temporarily injected to observe the subsequent execution path.
- The system returned access=true despite detecting knife:0.88.
- Internal details were also exposed.

Controlled path test

- Filename: ../logs/…..png.
- The test remained within the laboratory environment.
- The client-supplied path appeared in the response, and the file remained under the local directory used for testing.

The policy instrumentation was explicitly isolated. It does not replace the natural results of tests B and C; it makes it possible to reach the part of the code that would otherwise be hidden by the stream-handling bug.

7. Blue Team Defenses

lab_access_api_secure.py implements the following corrections:

- Request size limited to 1 MiB and image size limited to 900 KiB.
- A single bounded in-memory read, validated image decoding, and conversion of OpenCV errors into generic HTTP 400 responses.
- Resizing to 224 × 224, RGB conversion, float32 conversion, normalization to [0,1], and a batch shape of (1,224,224,3).
- Dangerous labels include knife, gun, weapon, weapon variants, and explosives. The confirmed threshold is 0.80, but any ambiguity still results in denial.
- Minimum facial confidence of 0.90; unknown labels and malformed outputs are rejected.
- UUID-generated filenames under logs/secure_uploads, with verification of the resolved parent directory.
- Minimal responses, with diagnostics available only on the server side.
- Pinned SHA-256 hashes and a restricted unpickler to maintain compatibility with the simulated models. In production, a non-executable format or an isolated service is preferable.
- Debug mode explicitly disabled, loopback as the default host, and generic error handling.

Each decision is documented directly in lab_access_api_secure.py and verified by the tests under tests/.

8. Retest

The results are stored in evidence/secure_controlled_retest_final_2.json. The regression test suite was executed using unittest: 9 tests, all 9 passed.

An initial assertion for a malformed request expected the multipart parser to raise an exception. However, Werkzeug treated the payload as an empty form, and the route correctly returned No image provided.

The initial failure is preserved in evidence/unit_test_output_final_9.txt. The test was corrected to explicitly exercise the HTTP 404 handler, and the corrected version passed in evidence/unit_test_output_final_9_fixed.txt.

Baseline vs. Secured Version

Test	Baseline	Secured Version
A — No image	HTTP 400	HTTP 400
B — Valid PNG	HTTP 500 before reaching the model	HTTP 200, access:false because knife:0.88 is rejected
C — Text file	OpenCV HTTP 500 / traceback in HTTP evidence	HTTP 400, generic message without a traceback
D — Request larger than 1 MiB	No configured limit	HTTP 413, request rejected
Preprocessing	(300,300,3) uint8	(1,224,224,3) float32, values in [0,1]
Response	Paths, IP address, scores, and labels	Only {"access":false}

	exposed in the isolated execution path	
Filename	Client-supplied path used	UUID filename directly under logs/secure_uploads
Models	Unrestricted pickle.load(), incompatible direct import	Verified hashes and authorized classes loaded

A direct network startup was also attempted in the sandbox but failed with Operation not permitted. This error is preserved in evidence/startup_probe_2026-10-25.txt.
Therefore, the retests reported here were performed using the local Flask test client, not through a claimed successful network listener.

9. Residual Risks and Recommendations

- Pickle remains inherently an executable serialization format. The current control is suitable for the two fixed laboratory artifacts, not for arbitrary models.

- The embedded hashes should be replaced with a signed and protected trust chain in a real deployment.

- Flask request size limits do not replace rate limiting, quotas, timeouts, authentication, authorization, and monitoring.

- The simulated models do not demonstrate biometric accuracy or robustness against adversarial transformations. A production deployment would require evaluation datasets, calibrated thresholds, human review, and dedicated security testing.

- The local demonstration log is not an immutable audit system. Appropriate permissions, log rotation, and retention policies must be implemented in production.

10. Conclusion

The PROVE phase confirmed weaknesses involving input validation, stream handling, model input contracts, dangerous-object policies, model loading, file paths, information disclosure, and configuration.

The DEFEND phase addresses these issues in a separate file without overwriting lab_access_api.py.

The RETEST phase confirms the rejection of knife, invalid and oversized input handling, the corrected tensor contract, and the reduction of response information.

Sandbox limitations and observed errors have been preserved rather than presented as successful operations.

Local References

- lab_access_api.py and generate_models.py: code provided or adapted from the laboratory.
- lab_access_api_secure.py: Blue Team implementation.
- evidence/baseline_controlled_run_final.json and evidence/secure_controlled_retest_final_2.json.
- evidence/before_after_comparison_final.json: machine-generated comparison of statuses.
- Materials provided under files/.